

E-COMMERCE WEBSITE

Activity Instructions and Grading Rubric

Name: _____

Score: _____ / 100

Course & Section: _____

Date: _____

I. Instructions

Create a fully functional e-commerce website using React. The website must have a product catalog, a shopping cart, and a checkout form — but it must be built **without a database, without a backend server, and without any real API calls** (no Axios/fetch to an external server). All product, cart, and order data should be created and managed entirely using `useState`, stored as JavaScript arrays and objects that live only in memory while the application is running.

It is expected that your data will reset every time the page is refreshed since there is no database involved — this is normal and acceptable. React Router may be used for navigating between pages, as this is only a front-end library and not a backend.

1. Build the project using React and Vite (JavaScript variant).
2. You may hardcode your starting product list directly in your code. Include at least 8 to 12 products.
3. You may choose any styling method: External CSS, Inline CSS, CSS Modules, Tailwind CSS, React Bootstrap, or Material UI.
4. Submit your complete source code (excluding the `node_modules` folder) or a link to your GitHub repository. Be ready to briefly explain your code, especially your cart and checkout logic, when asked.

II. Features to Include

Your e-commerce website must include all of the following features:

1. **Home Page / Product Listing** — displays all available products, each showing an image, name, price, and short description.
2. **Product Details Page** — shows the complete information of a selected product, such as a larger image, full description, price, and stock availability.
3. **Search and/or Category Filter** — allows customers to quickly find specific products by name or category.
4. **Shopping Cart** — customers can add products to the cart, view all items added, increase or decrease the quantity of an item, remove an item, and see the subtotal per item and the overall total.
5. **Cart Counter** — shows the current number of items in the cart, visible on every page.
6. **Checkout Page** — a form where the customer enters their Full Name, Email Address, Phone Number, Delivery Address, and Payment Method (Cash on Delivery).
7. **Form Validation** — the checkout form should not be submitted if required fields are left blank or filled out incorrectly (e.g., invalid email format, incomplete phone number), and clear error messages should be shown to the customer.
8. **Product Pagination or "View More"** — products are displayed in manageable groups or pages instead of showing everything at once.
9. **Navigation Menu** — a menu that allows the customer to move between the Home, Cart, and Checkout pages easily.
10. **Consistent Design** — the website should look clean, organized, and visually consistent across all pages.

III. Grading Rubric (100 Points)

Your output will be evaluated against the description that best matches your actual work.

Criteria	Pts	Excellent	Good	Fair	Poor
Project Setup & Component Architecture	10	10-9 — Project correctly set up with Vite; clear, logical folder/component structure; multiple well-separated, reusable components used throughout.	8-7 — Project set up correctly; component structure mostly organized with only minor issues.	6-5 — Project runs, but organization is inconsistent or several components are unnecessarily combined into one file.	4-0 — Setup is incomplete/broken, or the app has little to no separation into components.
JSX, Components & Routing	15	15-13 — JSX rules followed correctly (single parent element, closed tags, correct attribute names); at least 5 routes work smoothly via React Router; navigation links all function correctly.	12-10 — JSX mostly correct with only minor syntax issues; routing implemented with 3-4 working routes.	9-7 — Several JSX rule violations, or routing is only partially working (broken links/incorrect routes).	6-0 — JSX has major errors that break the app, or routing/navigation is missing entirely.
Props, PropTypes & Styling	15	15-13 — Product data is properly passed via props to reusable components; props validated with PropTypes (including required/default props); consistent, attractive styling across all components.	12-10 — Props used correctly, but PropTypes validation is incomplete; styling applied but not fully consistent.	9-7 — Limited use of props (data hardcoded in multiple components instead); little to no PropTypes or styling.	6-0 — Props not used correctly, causing components to display incorrect or no data.
Events & State Management (Cart)	20	20-18 — <code>useState</code> correctly manages product/cart arrays; Add to Cart, quantity update, and remove item all work and update the UI in real time; totals compute correctly.	17-14 — Cart functionality works with minor bugs (e.g., total not updating in one case, or slight UI delay).	13-10 — Only partial cart functionality (e.g., can add but not remove/update quantity), or state updates are unreliable.	9-0 — Cart functionality does not work, or <code>useState</code> /events are implemented incorrectly.
Advanced Component	15	15-13 — Pagination and/or a UI library (MUI/Bootstrap) and/or an advanced table is implemented correctly and clearly improves the user experience.	12-10 — At least one advanced component is implemented, with only minor issues.	9-7 — An advanced component is attempted but does not function correctly.	6-0 — No advanced component (pagination/library/table) was implemented.
Forms & Validation (Checkout)	15	15-13 — Checkout form uses correct, fully controlled form elements; both HTML validation and custom JavaScript validation are implemented with clear error messages; successful submission clears the cart.	12-10 — Form works and validates, but limited to only HTML attributes or only one custom rule.	9-7 — Form is present but not fully controlled by state, or validation is minimal/missing.	6-0 — No functional checkout form, or the form does not submit/validate at all.
Overall Functionality, Code Quality & Creativity	10	10-9 — App runs from start to finish (browse, add to cart, checkout) with no console errors; code is clean and well-organized; UI/UX shows creativity and effort.	8-7 — App is mostly functional with minor bugs; code is generally readable.	6-5 — App has noticeable bugs or an incomplete flow; code is messy or hard to follow.	4-0 — App does not run, or crashes/major errors prevent proper evaluation.
TOTAL		100 POINTS			

Note: A student or group may be asked to briefly explain their code (a short oral defense) to verify original understanding of the concepts, especially the state and props logic used for the cart and checkout.

Prepared by: _____